

Lecture # 02

Flynn's Taxonomy: Types of Parallelism

PARALLEL AND DISTRIBUTED COMPUTING

CS-482



Flynn's Taxonomy:

Parallel computing is computing where the jobs are broken into discrete parts that can be executed concurrently. Each part is further broken down into a series of instructions. Instructions from each piece execute simultaneously on different CPUs. The breaking up of different parts of a task among multiple processors will help to reduce the amount of time to run a program. Parallel systems deal with the simultaneous use of multiple computer resources that can include a single computer with multiple processors, a number of computers connected by a network to form a parallel processing cluster, or a combination of both. Parallel systems are more difficult to program than computers with a single processor because the architecture of parallel computers varies accordingly and the processes of multiple CPUs must be coordinated and synchronized. The difficult problem of parallel processing is portability.

Flynn's taxonomy

An Instruction Stream is a sequence of instructions that are read from memory. Data Stream is the operations performed on the data in the processor. Flynn's taxonomy is a classification scheme for computer architectures proposed by Michael Flynn in 1966. The taxonomy is based on the number of instruction streams and data streams that can be processed simultaneously by a computer architecture.

Flynn's taxonomy

There are four categories in Flynn's taxonomy:

1. Single Instruction Single Data (SISD): In a SISD architecture, there is a single processor that executes a single instruction stream and operates on a single data stream. This is the simplest type of computer architecture and is used in most traditional computers.
2. Single Instruction Multiple Data (SIMD): In a SIMD architecture, there is a single processor that executes the same instruction on multiple data streams in parallel. This type of architecture is used in applications such as image and signal processing.
3. Multiple Instruction Single Data (MISD): In a MISD architecture, multiple processors execute different instructions on the same data stream. This type of architecture is not commonly used in practice, as it is difficult to find applications that can be decomposed into independent instruction streams.
4. Multiple Instruction Multiple Data (MIMD): In a MIMD architecture, multiple processors execute different instructions on different data streams. This type of architecture is used in distributed computing, parallel processing, and other high-performance computing applications.

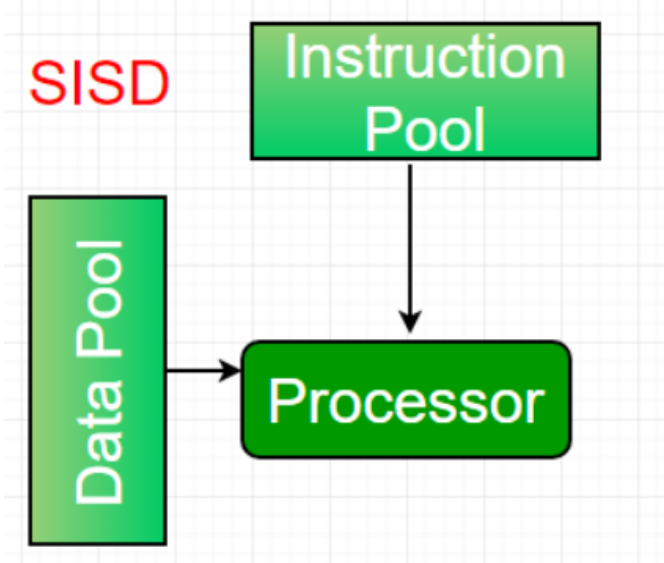
Flynn's taxonomy

Flynn's taxonomy is a useful tool for understanding different types of computer architectures and their strengths and weaknesses. The taxonomy highlights the importance of parallelism in modern computing and shows how different types of parallelism can be exploited to improve performance.

		Instruction Streams	
		one	many
Data Streams	one	SISD traditional von Neumann single CPU computer	MISD May be pipelined Computers
	many	SIMD Vector processors fine grained data Parallel computers	MIMD Multi computers Multiprocessors

Flynn's classification

1. Single-instruction, single-data (SISD) systems – An SISD computing system is a uniprocessor machine that is capable of executing a single instruction, operating on a single data stream. In SISD, machine instructions are processed in a sequential manner and computers adopting this model are popularly called sequential computers. Most conventional computers have SISD architecture. All the instructions and data to be processed have to be stored in primary memory.

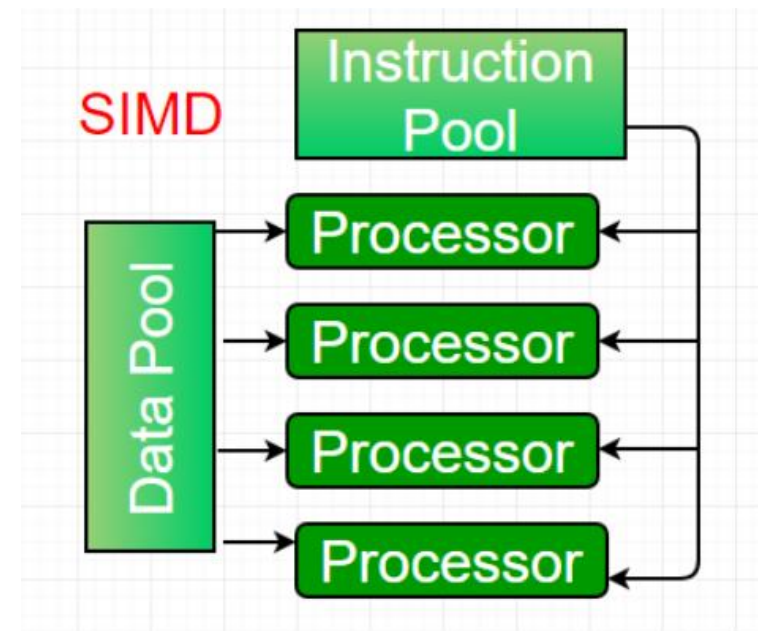


SISD

The speed of the processing element in the SISD model is limited(dependent) by the rate at which the computer can transfer information internally. Dominant representative SISD systems are IBM PC, workstations.

Single-instruction, multiple-data (SIMD) systems

A SIMD system is a multiprocessor machine capable of executing the same instruction on all the CPUs but operating on different data streams. Machines based on a SIMD model are well suited to scientific computing since they involve lots of vector and matrix operations. So that the information can be passed to all the processing elements (PEs) organized data elements of vectors can be divided into multiple sets (N-sets for N PE systems) and each PE can process one data set.



SIMD

Dominant representative SIMD systems is Cray's vector processing machine.

Flynn's 1972 paper subdivided SIMD down into three further categories:

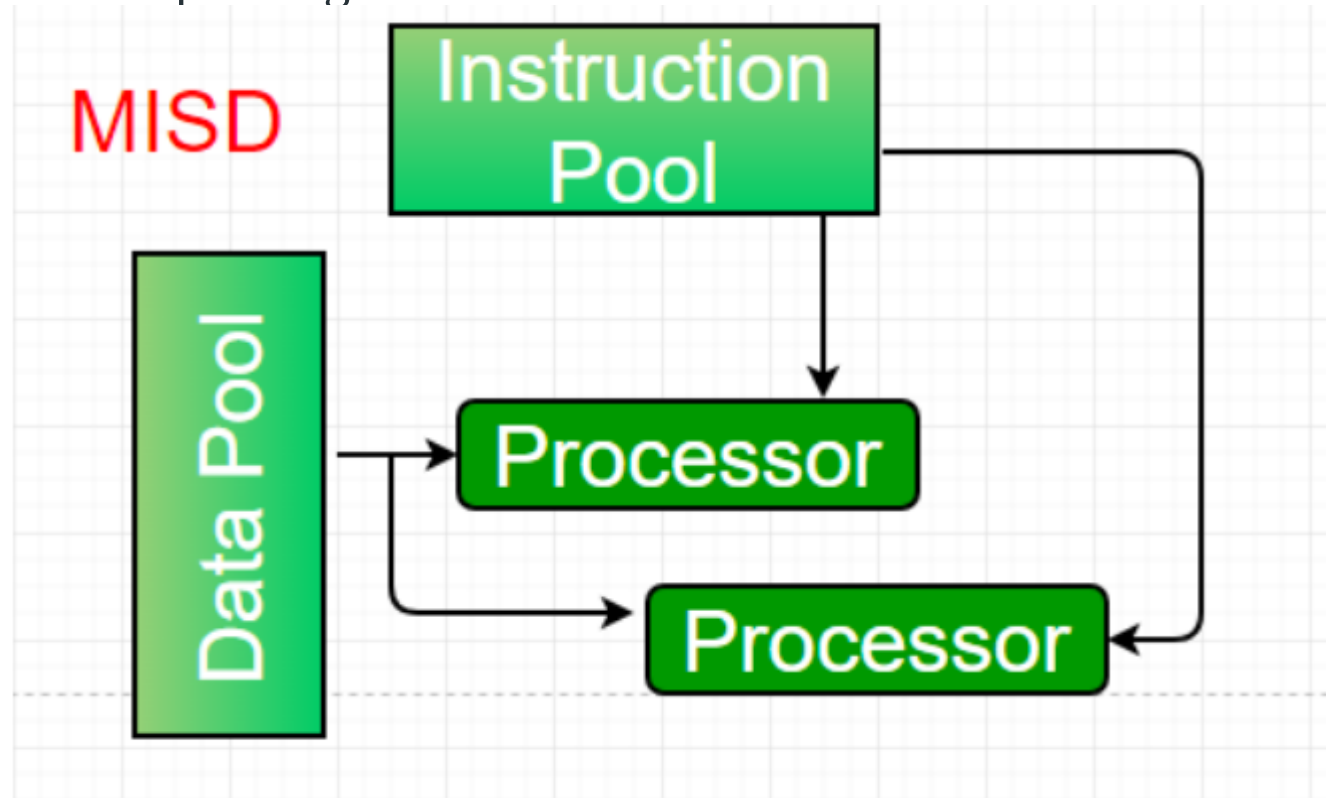
- **Array processor**
- **Pipelined processor**
- **Associative processor**

SIMD sub categories

- **Array processor** – These receive the one (same) instruction but each parallel processing unit has its own separate and distinct memory and register file.
- **Pipelined processor** – These receive the one (same) instruction but then read data from a central resource, each processes fragments of that data, then writes back the results to the same central resource. In Figure 5 of Flynn's 1972 paper that resource is main memory: for modern CPUs that resource is now more typically the register file.
- **Associative processor** – These receive the one (same) instruction but in each parallel processing unit an *independent* decision is made, based on data *local* to the unit, as to whether to perform the execution or whether to skip it. In modern terminology this is known as "predicated" (masked) SIMD.

Multiple-instruction, single-data (MISD) systems

An MISD computing system is a multiprocessor machine capable of executing different instructions on different PEs but all of them operating on the same dataset .



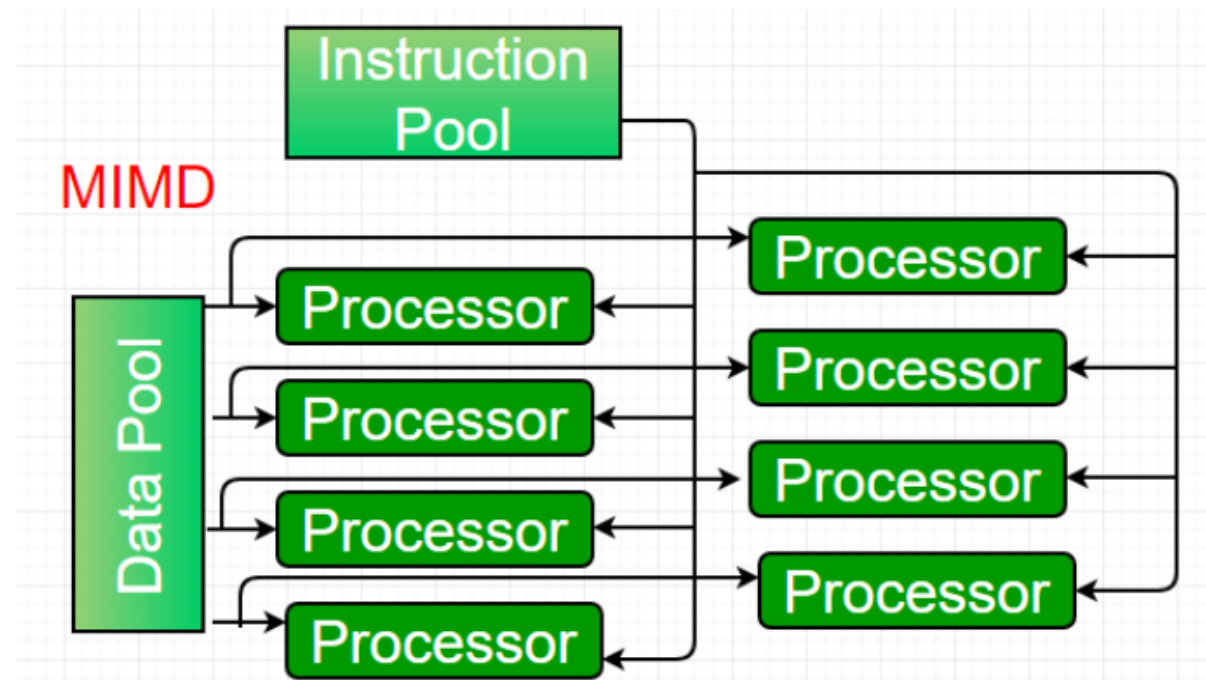
MISD

Example $Z = \sin(x) + \cos(x) + \tan(x)$

The system performs different operations on the same data set. Machines built using the MISD model are not useful in most of the application, a few machines are built, but none of them are available commercially.

Multiple-instruction, multiple-data (MIMD) systems

An MIMD system is a multiprocessor machine which is capable of executing multiple instructions on multiple data sets. Each PE in the MIMD model has separate instruction and data streams; therefore, machines built using this model are capable to any kind of application. Unlike SIMD and MISD machines, PEs in MIMD machines work asynchronously.



MIMD

MIMD machines are broadly categorized into **shared-memory MIMD** and **distributed-memory MIMD** based on the way PEs are coupled to the main memory.

Shared memory MIMD

In the **shared memory MIMD** model (tightly coupled multiprocessor systems), all the PEs are connected to a single global memory and they all have access to it. The communication between PEs in this model takes place through the shared memory, modification of the data stored in the global memory by one PE is visible to all other PEs. Dominant representative shared memory MIMD systems are Silicon Graphics machines and Sun/IBM's SMP (Symmetric Multi-Processing).

Distributed memory MIMD

In **Distributed memory MIMD** machines (loosely coupled multiprocessor systems) all PEs have a local memory. The communication between PEs in this model takes place through the interconnection network (the inter process communication channel, or IPC). The network connecting PEs can be configured to tree, mesh or in accordance with the requirement.

MIMD architectures

The shared-memory MIMD architecture is easier to program but is less tolerant to failures and harder to extend with respect to the distributed memory MIMD model. Failures in a shared-memory MIMD affect the entire system, whereas this is not the case of the distributed model, in which each of the PEs can be easily isolated. Moreover, shared memory MIMD architectures are less likely to scale because the addition of more PEs leads to memory contention. This is a situation that does not happen in the case of distributed memory, in which each PE has its own memory. As a result of practical outcomes and user's requirement , distributed memory MIMD architecture is superior to the other existing models.

SPMD and MPMD

SPMD and MPMD: is a programming model or execution paradigm that applies specifically to MIMD architectures.

It describes how parallel tasks are organized and executed, but it is not a hardware classification.

- **SPMD** assumes all processors execute the same program on different data.
- **MPMD** assumes processors execute different programs on different data.

SPMD

Single Program, Multiple Data Streams (SPMD):

SPMD is a **parallel execution model** in which:

- Multiple autonomous processors execute the **same program** on **different data sets**.
- Unlike SIMD (Single Instruction, Multiple Data), processors in SPMD **do not operate in lockstep**. Each processor executes the program independently and can take different execution paths based on the data it processes.
- SPMD is widely used in High-Performance Computing (HPC), particularly for tasks like distributed computing, where the same algorithm (program) runs across multiple nodes working on different chunks of a data set.

Example:

- A distributed machine learning model training system where each node processes a subset of the training data but uses the same algorithm.
- Federica Darema originally proposed SPMD as part of the RP3 project.

MPMD

Multiple Programs, Multiple Data Streams (MPMD):

MPMD is a model in which:

- **Multiple autonomous processors execute multiple different programs** simultaneously, each working on **different data sets**.
- Often used in heterogeneous computing systems, where one processor acts as a "manager" or "host" and other processors act as "workers."
- The manager program distributes tasks to worker programs, which then operate independently on their data.

Types of parallelism

1. Hardware-Level Parallelism

- **Parallelism in a Uniprocessor:**

Pipelining Superscalar architectures, Very Long Instruction Word (VLIW) architectures, etc.

- **SIMD Instructions and Processing:**

Vector processors GPUs

- **Multiprocessor Systems:**

Symmetric shared-memory multiprocessors

Distributed-memory multiprocessors

Chip-multiprocessors (Multi-core systems)

- **Multi-computers (Clusters):**

Systems consisting of multiple independent computers working together.

Software-Level Parallelism

- **Data-Level Parallelism**
- **Instruction-Level Parallelism**
- **Task-Level Parallelism**

Introduction to Parallel Computing

Before taking a toll on Parallel Computing, first, let's take a look at the background of computations of computer software and why it failed for the modern era.

Computer software was written conventionally for serial computing. This meant that to solve a problem, an algorithm divides the problem into smaller instructions. These discrete instructions are then executed on the Central Processing Unit of a computer one by one. Only after one instruction is finished, next one starts.

A real-life example of this would be people standing in a queue waiting for a movie ticket and there is only a cashier. The cashier is giving tickets one by one to the persons. The complexity of this situation increases when there are 2 queues and only one cashier.

Introduction to Parallel Computing

So, in short, Serial Computing is following:

1. In this, a problem statement is broken into discrete instructions.
2. Then the instructions are executed one by one.
3. Only one instruction is executed at any moment of time.

Look at point 3. This was causing a huge problem in the computing industry as only one instruction was getting executed at any moment of time. This was a huge waste of hardware resources as only one part of the hardware will be running for particular instruction and of time. As problem statements were getting heavier and bulkier, so does the amount of time in execution of those statements. Examples of processors are Pentium 3 and Pentium 4.

Need for Parallel Computing

Now let's come back to our real-life problem. We could definitely say that complexity will decrease when there are 2 queues and 2 cashiers giving tickets to 2 persons simultaneously. This is an example of Parallel Computing.

Parallel Computing :

It is the use of multiple processing elements simultaneously for solving any problem. Problems are broken down into instructions and are solved concurrently as each resource that has been applied to work is working at the same time.

Advantages of Parallel Computing

Advantages of Parallel Computing over Serial Computing are as follows:

1. It saves time and money as many resources working together will reduce the time and cut potential costs.
2. It can be impractical to solve larger problems on Serial Computing.
3. It can take advantage of non-local resources when the local resources are finite.
4. Serial Computing ‘wastes’ the potential computing power, thus Parallel Computing makes better work of the hardware.

Instruction-level parallelism

A processor can only address less than one instruction for each clock cycle phase. These instructions can be re-ordered and grouped which are later on executed concurrently without affecting the result of the program. This is called instruction-level parallelism.

Task Parallelism

Task parallelism employs the decomposition of a task into subtasks and then allocating each of the subtasks for execution. The processors perform the execution of sub-tasks concurrently.

Data-level parallelism (DLP)

Instructions from a single stream operate concurrently on several data – Limited by non-regular data manipulation patterns and by memory bandwidth

Why parallel computing?

- The whole real-world runs in dynamic nature i.e. many things happen at a certain time but at different places concurrently. This data is extensively huge to manage.
- Real-world data needs more dynamic simulation and modeling, and for achieving the same, parallel computing is the key.
- Parallel computing provides concurrency and saves time and money.
- Complex, large datasets, and their management can be organized only and only using parallel computing's approach.
- Ensures the effective utilization of the resources. The hardware is guaranteed to be used effectively whereas in serial computation only some part of the hardware was used and the rest rendered idle.
- Also, it is impractical to implement real-time systems using serial computing.

Applications of Parallel Computing:

- Databases and Data mining.
- Real-time simulation of systems.
- Science and Engineering.
- Advanced graphics, augmented reality, and virtual reality.

Limitations of Parallel Computing:

- It addresses such as communication and synchronization between multiple sub-tasks and processes which is difficult to achieve.
- The algorithms must be managed in such a way that they can be handled in a parallel mechanism.
- The algorithms or programs must have low coupling and high cohesion. But it's difficult to create such programs.
- More technically skilled and expert programmers can code a parallelism-based program well.

Future of Parallel Computing:

The computational graph has undergone a great transition from serial computing to parallel computing. Tech giant such as Intel has already taken a step towards parallel computing by employing multicore processors. Parallel computation will revolutionize the way computers work in the future, for the better good. With all the world connecting to each other even more than before, Parallel Computing does a better role in helping us stay that way. With faster networks, distributed systems, and multi-processor computers, it becomes even more necessary.

Task#01

Flynn's Taxonomy classifies computer architectures into four categories: SISD, SIMD, MISD, and MIMD. Analyze the following scenarios and classify each into the appropriate category. Justify your classification for each case:

- A basic single-core processor executes a single instruction on a single data stream, such as adding two numbers.
- A GPU processes large-scale matrix multiplication by performing the same operation (e.g., addition) on multiple elements simultaneously.
- A fault-tolerant system executes multiple instructions on the same data stream to verify the results through redundancy and comparison.
- A cluster of distributed systems performs different tasks, such as running multiple machine learning models on separate datasets simultaneously.

Task#02

Which architecture is most suitable for:

- Real-time graphics rendering?
- Large-scale parallel scientific simulations?

Explain why MISD is rarely implemented in modern computer architectures.

Task#03

A multi-core processor is evaluated for its efficiency under different classifications of Flynn's taxonomy: SISD, SIMD, MISD, and MIMD.

Scenario:

1. **SISD (Single Instruction Single Data):** The processor executes a single instruction on a single data stream. It processes 1 instruction per clock cycle.
2. **SIMD (Single Instruction Multiple Data):** The processor executes the same instruction on multiple data streams in parallel. It processes 4 data streams per clock cycle.
3. **MIMD (Multiple Instruction Multiple Data):** The processor executes multiple instructions on multiple data streams. It processes 8 instructions on 8 data streams per clock cycle.
4. **MISD (Multiple Instruction Single Data):** The processor executes multiple instructions on a single data stream. This architecture processes 3 instructions on the same data stream per clock cycle.

Question:

A computational task requires processing **10,000 data units**, with:

- Each instruction requiring **5 clock cycles** for completion.

Determine:

1. Total clock cycles required for the task under each classification.
2. Identify the most efficient architecture in terms of processing time.